Introduction to Haskell Programming

Exploring graphs

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Lazy evaluation

- Recall that Haskell uses lazy evaluation

Lazy evaluation

- Recall that Haskell uses lazy evaluation
- Outermost reduction

Lazy evaluation

- Recall that Haskell uses lazy evaluation
- Outermost reduction
- Simplify function definition first

Lazy evaluation

- Recall that Haskell uses lazy evaluation
- Outermost reduction
- Simplify function definition first
- Compute argument value only if needed

Graphs

- A graph G is defined by a set of vertices V and a set of (directed) edges $E \subseteq V \times V$

Graphs

- A graph G is defined by a set of vertices V and a set of (directed) edges $E \subseteq V \times V$
- Assume that each vertex is represented by a character

Graphs

- A graph G is defined by a set of vertices V and a set of (directed) edges $E \subseteq V \times V$
- Assume that each vertex is represented by a character
- In Haskell:

```
type Vertex = Char
type Edge = (Vertex, Vertex)
vertices :: [Vertex]
edges :: [Edge]
```

Graphs

- An example graph:

```
vertices :: [Vertex]
vertices = "abcdef"

edges :: [Edge]
edges = [('a','b'), ('a','d'), ('b','c'), ('f','d'),
         ('c','a'), ('c','e'), ('d','e'), ('f','e')]

edge :: Vertex → Vertex → Bool
edge u v = (u,v) `elem` edges
```

Connectedness

- `connected :: Vertex → Vertex → Bool`

Connectedness

- `connected :: Vertex → Vertex → Bool`
- `connected u v` is **`True`** iff there is a path from `u` to `v` using the given set of edges

Connectedness

- `connected :: Vertex → Vertex → Bool`
- `connected u v` is **`True`** iff there is a path from `u` to `v` using the given set of edges
- Inductive definition:

Connectedness

- `connected :: Vertex → Vertex → Bool`
- `connected u v` is **`True`** iff there is a path from `u` to `v` using the given set of edges
- Inductive definition:
  - If `edge u v` then `connected u v`

Connectedness

- `connected :: Vertex → Vertex → Bool`
- `connected u v` is **`True`** iff there is a path from `u` to `v` using the given set of edges
- Inductive definition:
  - If `edge u v` then `connected u v`
  - If `connected u v` and `edge v w`, then `connected u w`

Connectedness

- `connected :: Vertex → Vertex → Bool`
- `connected u v` is **`True`** iff there is a path from `u` to `v` using the given set of edges
- Inductive definition:
  - If `edge u v` then `connected u v`
  - If `connected u v` and `edge v w`, then `connected u w`
- Difficult to translate this directly to Haskell

Connectedness

- To check if `connected u v`:

Connectedness

- To check if `connected u v`:
  - Build all paths in the graph

Connectedness

- To check if `connected u v`:
  - Build all paths in the graph
  - Check if there is a path with first vertex `u` and last vertex `v`

Connectedness

- To check if `connected u v`:
  - Build all paths in the graph
  - Check if there is a path with first vertex `u` and last vertex `v`
- The set of all paths is infinite

Connectedness

- To check if `connected u v`:
  - Build all paths in the graph
  - Check if there is a path with first vertex `u` and last vertex `v`
- The set of all paths is infinite
- But Haskell's laziness ensures termination!

Building paths

- Build paths by induction on length (number of vertices on path)

Building paths

- Build paths by induction on length (number of vertices on path)
- Only one path of length `0` (the empty path)

Building paths

- Build paths by induction on length (number of vertices on path)
- Only one path of length `0` (the empty path)
- Extend path of length `k` to `k+1` by adding an edge

```
type Path = [Vertex]
extendPath :: Path → [Path]
extendPath []       = [ [v] | v ← vertices]
extendPath (v:vs)   = [u:v:vs | u ← vertices, edge u v]
```

Building paths

- `extendAll ps` extends all paths in `ps` by adding an edge

```
extendAll :: [Path] → [Path]
extendAll = concatMap extendPath
```

Building paths

- `extendAll ps` extends all paths in `ps` by adding an edge

```
extendAll :: [Path] → [Path]
extendAll = concatMap extendPath
```

- Built-in function `iterate`

```
iterate :: (a → a) → a → [a]
iterate f x = x:iterate f (f x)
iterate f x ---> [x, f x, f (f x), f (f (f x)), ...]
```

Building paths

- `extendAll ps` extends all paths in `ps` by adding an edge

```
extendAll :: [Path] → [Path]
extendAll = concatMap extendPath
```

- Built-in function `iterate`

```
iterate :: (a → a) → a → [a]
iterate f x = x:iterate f (f x)
iterate f x ---> [x, f x, f (f x), f (f (f x)), ...]
```

- All paths:

```
allPaths :: [[Path]]
allPaths = iterate extendAll [[]]
```

Connectivity

- All paths:

```
allPaths :: [[Path]]
allPaths = iterate extendAll [[]]
```

Connectivity

- All paths:

```
allPaths :: [[Path]]
allPaths = iterate extendAll [[]]
```

- If there is a path from u to v, there is a path with at most n edges

```
shortPaths = concat $ take (length vertices + 1) allPaths
```

Connectivity

- All paths:

```
allPaths :: [[Path]]
allPaths = iterate extendAll [[]]
```

- If there is a path from u to v, there is a path with at most n edges

```
shortPaths = concat $ take (length vertices + 1) allPaths
```

- All connected pairs:

```
connectedPairs = [(u,v) | u:p ← shortPaths, v ← u:p]
```

Connectivity

- All paths:

```
allPaths :: [[Path]]
allPaths = iterate extendAll [[]]
```

- If there is a path from u to v, there is a path with at most n edges

```
shortPaths = concat $ take (length vertices + 1) allPaths
```

- All connected pairs:

```
connectedPairs = [(u,v) | u:p ← shortPaths, v ← u:p]
```

- Connectivity:

```
connected u v = (u,v) `elem` connectedPairs
```

Search problems

- No closed form for the solution

Search problems

- No closed form for the solution
- Go through a cycle of:

Search problems

- No closed form for the solution
- Go through a cycle of:
  - expanding out possible solutions

Search problems

- No closed form for the solution
- Go through a cycle of:
  - expanding out possible solutions
  - undoing partial solutions when we reach a dead end

Search problems

- No closed form for the solution
- Go through a cycle of:
  - expanding out possible solutions
  - undoing partial solutions when we reach a dead end
- **Backtracking**

The n queens problem

- Place n queens on $n \times n$ chessboard so that no two attack each other

The n queens problem

- Place n queens on $n \times n$ chessboard so that no two attack each other
- Two queens attack each other if they are on the same row, column or diagonal

The n queens problem

- Place n queens on $n \times n$ chessboard so that no two attack each other
- Two queens attack each other if they are on the same row, column or diagonal
- n queens, n rows, n columns

The n queens problem

- Place n queens on $n \times n$ chessboard so that no two attack each other
- Two queens attack each other if they are on the same row, column or diagonal
- n queens, n rows, n columns
  - Exactly one queen in each row, each column

Heuristic

- Place the first queen somewhere in row $n - 1$

Heuristic

- Place the first queen somewhere in row $n - 1$
- For each lower row, place a queen at the leftmost square that is not attacked by any of the earlier queens

Heuristic

- Place the first queen somewhere in row $n - 1$
- For each lower row, place a queen at the leftmost square that is not attacked by any of the earlier queens
- If the current configuration cannot be extended to n queens, try the next available square

Heuristic

- Place the first queen somewhere in row $n - 1$
- For each lower row, place a queen at the leftmost square that is not attacked by any of the earlier queens
- If the current configuration cannot be extended to n queens, try the next available square
- If you run out of squares in this row, try another square on the previous row

Heuristic

- Place the first queen somewhere in row $n - 1$
- For each lower row, place a queen at the leftmost square that is not attacked by any of the earlier queens
- If the current configuration cannot be extended to n queens, try the next available square
- If you run out of squares in this row, try another square on the previous row
- **Backtracking**

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where `0 ⩽ i ⩽ n`)

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where `0 ⩽ i ⩽ n`)
- Stores column numbers of queens in rows `i, ... ,n-1`

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where $0 \leqslant i \leqslant n$)
- Stores column numbers of queens in rows `i, ... ,n-1`
- … in `conf !! 0, ... ,conf !! (n-i)`

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where `0 ⩽ i ⩽ n`)
- Stores column numbers of queens in rows `i, ... ,n-1`
- … in `conf !! 0, ... ,conf !! (n-i)`
- So `conf !! j` corresponds to row `n - length conf + j`

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where `0 ⩽ i ⩽ n`)
- Stores column numbers of queens in rows `i, ... ,n-1`
- … in `conf !! 0, ... ,conf !! (n-i)`
- So `conf !! j` corresponds to row `n - length conf + j`
- Adding a new queen in row `i-1` corresponds to adding an element at the head of `conf`

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where `0 ⩽ i ⩽ n`)
- Stores column numbers of queens in rows `i, ... ,n-1`
- … in `conf !! 0, ... ,conf !! (n-i)`
- So `conf !! j` corresponds to row `n - length conf + j`
- Adding a new queen in row `i-1` corresponds to adding an element at the head of `conf`
- **Check that the index math works correctly!**

n queens in Haskell

- Arrangement of queens is a list

```
type Config = [Int]
```

- A configuration `conf` is a list of length `n-i` (where `0 ⩽ i ⩽ n`)
- Stores column numbers of queens in rows `i, ... ,n-1`
- … in `conf !! 0, ... ,conf !! (n-i)`
- So `conf !! j` corresponds to row `n - length conf + j`
- Adding a new queen in row `i-1` corresponds to adding an element at the head of `conf`
- **Check that the index math works correctly!**
- Example configuration for eight queens: `[0,2,4,6,1,3,5]` – **cannot be extended!**

Extending a configuration

- `extendByOne` – function to extend a configuration by adding one more queen

```
extendByOne :: Int → Config → [Config]
```

Extending a configuration

- `extendByOne` – function to extend a configuration by adding one more queen

```
extendByOne :: Int → Config → [Config]
```

  - `extendByOne n conf` gives all ways of extending `conf`

Extending a configuration

- `extendByOne` – function to extend a configuration by adding one more queen

```
extendByOne :: Int → Config → [Config]
```

  - `extendByOne n conf` gives all ways of extending `conf`
  - New queen is in a column from `[0..(n-1)]`

Extending a configuration

- `extendByOne` – function to extend a configuration by adding one more queen

```
extendByOne :: Int → Config → [Config]
```

  - `extendByOne n conf` gives all ways of extending `conf`
  - New queen is in a column from `[0..(n-1)]`
- The new queen:

Extending a configuration

- `extendByOne` – function to extend a configuration by adding one more queen

```
extendByOne :: Int → Config → [Config]
```

  - `extendByOne n conf` gives all ways of extending `conf`
  - New queen is in a column from `[0..(n-1)]`
- The new queen:
  - should not be in same column as an earlier queen

Extending a configuration

- `extendByOne` – function to extend a configuration by adding one more queen

```
extendByOne :: Int → Config → [Config]
```

  - `extendByOne n conf` gives all ways of extending `conf`
  - New queen is in a column from `[0..(n-1)]`
- The new queen:
  - should not be in same column as an earlier queen
  - should not be on same diagonal or anti-diagonal as an earlier queen

Extending a configuration

- Code to extend a configuration by one row (below all the filled rows)

```
extendByOne          :: Int → Config → [Config]
extendByOne n conf   = [c:conf | c ← [0..(n-1)], safe c conf] where
    safe c conf      =   c `notElem` [ j     | (i,j) ← occupiedCells]
                       && c `notElem' [ j-i   | (i,j) ← occupiedCells]
                       && c `notElem' [ j+i   | (i,j) ← occupiedCells]
    occupiedCells    = zip [1..] conf
```

Extending a configuration

- Code to extend a configuration by one row (below all the filled rows)

```
extendByOne          :: Int → Config → [Config]
extendByOne n conf   = [c:conf | c ← [0..(n-1)], safe c conf] where
    safe c conf      =   c `notElem` [ j     | (i,j) ← occupiedCells]
                       && c `notElem' [ j-i   | (i,j) ← occupiedCells]
                       && c `notElem' [ j+i   | (i,j) ← occupiedCells]
    occupiedCells    = zip [1..] conf
```

- `(0,c)` is the position of the new queen

Extending a configuration

- Code to extend a configuration by one row (below all the filled rows)

```
extendByOne          :: Int → Config → [Config]
extendByOne n conf   = [c:conf | c ← [0..(n-1)], safe c conf] where
    safe c conf      =   c `notElem` [ j     | (i,j) ← occupiedCells]
                       && c `notElem` [ j-i   | (i,j) ← occupiedCells]
                       && c `notElem` [ j+i   | (i,j) ← occupiedCells]
    occupiedCells    = zip [1..] conf
```

- `(0,c)` is the position of the new queen
- It will clash with an existing queen at position `(i,j)` if:

Extending a configuration

- Code to extend a configuration by one row (below all the filled rows)

```
extendByOne          :: Int → Config → [Config]
extendByOne n conf   = [c:conf | c ← [0..(n-1)], safe c conf] where
    safe c conf      =   c `notElem` [ j     | (i,j) ← occupiedCells]
                       && c `notElem' [ j-i   | (i,j) ← occupiedCells]
                       && c `notElem' [ j+i   | (i,j) ← occupiedCells]
    occupiedCells    = zip [1..] conf
```

- `(0,c)` is the position of the new queen
- It will clash with an existing queen at position `(i,j)` if:
  - either `c == j`, meaning the queens are in the same column

Extending a configuration

- Code to extend a configuration by one row (below all the filled rows)

```
extendByOne          :: Int → Config → [Config]
extendByOne n conf   = [c:conf | c ← [0..(n-1)], safe c conf] where
    safe c conf      =   c `notElem` [ j     | (i,j) ← occupiedCells]
                       && c `notElem` [ j-i   | (i,j) ← occupiedCells]
                       && c `notElem` [ j+i   | (i,j) ← occupiedCells]
    occupiedCells    = zip [1..] conf
```

- `(0,c)` is the position of the new queen
- It will clash with an existing queen at position `(i,j)` if:
  - either `c == j`, meaning the queens are in the same column
  - or `c-0 == j-i`, meaning the queens are in the same diagonal

Extending a configuration

- Code to extend a configuration by one row (below all the filled rows)

```
extendByOne          :: Int → Config → [Config]
extendByOne n conf   = [c:conf | c ← [0..(n-1)], safe c conf] where
    safe c conf      =   c `notElem` [ j     | (i,j) ← occupiedCells]
                       && c `notElem' [ j-i   | (i,j) ← occupiedCells]
                       && c `notElem' [ j+i   | (i,j) ← occupiedCells]
    occupiedCells    = zip [1..] conf
```

- `(0,c)` is the position of the new queen
- It will clash with an existing queen at position `(i,j)` if:
  - either `c == j`, meaning the queens are in the same column
  - or `c-0 == j-i`, meaning the queens are in the same diagonal
  - or `c+0 == j+i`, meaning the queens are in the same anti-diagonal

n queens

```
extend :: Int → Config → [Config]
extend n conf = if length conf == n
                then [conf]
                else concatMap (extend n) (extendByOne n conf)

queens :: Int → [Config]
queens n = extend n []

queensone :: Int → Config
queensone i = queens i !! 0

queensone 8 ---> [3,1,6,2,5,7,4,0]
```

Summary

- Infinite lists are a useful way to think about search problems

Summary

- Infinite lists are a useful way to think about search problems
- Instead of backtracking, iteratively generate all valid solutions upto a desired depth

Summary

- Infinite lists are a useful way to think about search problems
- Instead of backtracking, iteratively generate all valid solutions upto a desired depth
- Lazy evaluation converts inefficient breadth-first search to depth-first evaluation of leftmost solution